Graph Construction for Early Sepsis Prediction: Comparing Physiological and Data-Driven Relationships Across Hospitals

Nick Allen
Department of Computer Science
Stanford University
nallen21@stanford.edu

Chantal Le
Department of Computer Science
Stanford University
chantal8@stanford.edu

Tyler Yamori-Little
Department of Electrical Engineering
Stanford University
tyamori@stanford.edu

1 Introduction

Sepsis is a life-threatening condition characterized by organ dysfunction in response to infection, making early detection critical for timely treatment. Predicting sepsis from electronic health records (EHRs) is challenging because physiological measurements are irregularly sampled, frequently missing, and collected differently across hospitals. While conventional machine learning models capture temporal trends, graph neural networks (GNNs) can explicitly model relationships among physiological variables.

We will investigate how graph construction affects early sepsis prediction and cross-hospital generalization by comparing clinically informed, statistically inferred, and hybrid graphs. We hypothesize that clinically informed graphs may generalize better than hospital-specific statistical graphs, particularly when measurement practices differ across institutions. Our central questions are: (1) How does graph structure affect sepsis prediction? (2) Do clinically informed or hybrid graphs improve cross-hospital generalization?

2 Dataset and Prediction Task

We use the publicly available PhysioNet/Computing in Cardiology Challenge 2019 dataset [1], containing 40,336 ICU patient records from two hospital systems (20,336 from Hospital A and 20,000 from Hospital B) and 1,552,210 hourly observations. Each record includes eight vital signs, 26 laboratory measurements, and six demographic or contextual variables. Our task is to predict hourly sepsis risk using only information available up to the current hour, emphasizing detection approximately six hours before Sepsis-3 onset. The released label already turns on six hours before onset, and we do not shift it.

Only 7.27% of patients develop sepsis (1.80% of hours), and 80.1% of physiological entries are missing. Measurement practices also differ substantially: end-tidal CO_2 is absent in Hospital A, while bicarbonate and base excess are rarely measured in Hospital B. Because statistical relationships estimated from observed measurements may reflect hospital-specific testing practices, we investigate whether clinically informed graphs provide more transferable representations.

We split whole stays 70/15/15 into patient-disjoint training, validation (dev), and test sets of 28,235 / 6,053 / 6,048 patients, stratified by hospital, ever-septic status, and stay length. Our primary metric is

36th Conference on Neural Information Processing Systems (NeurIPS 2022).

normalized challenge utility, the official evaluation metric from the 2019 PhysioNet Sepsis Challenge, which rewards early detection (ideally six hours before onset) while penalizing false alarms and delayed predictions; never alarming scores 0 and the ideal alarm sequence scores 1. Secondary metrics include area under the precision-recall curve (AUPRC), AUROC, hourly sensitivity, and false-positive alarms per 100 non-septic patient-hours. For every model, the checkpoint epoch and alarm threshold are chosen on dev and frozen before the test set is scored.

3 Approach

We represent each patient-hour as a graph $G = (V, E)$, where each of the 34 nodes represents a distinct physiological variable (e.g., heart rate, blood pressure, or creatinine), and edges represent relationships between variables. Graph topology is shared across patients within each construction strategy, while node features vary by patient and time. Node i at hour t has three features: its most recent observed value (forward-filled from past hours only, or the training median if never measured, then standardized with training statistics), an observation mask indicating whether it was measured at hour t, and the time since its last measurement, capped at 24 hours. Demographic and contextual variables (age, sex, ICU unit, hospital-to-ICU admission time, and current ICU hour) enter the prediction head separately.

We compare the following graph construction strategies:

- **No edges (self-loops only).** Each variable attends only to itself. This is our model without message passing, with the same architecture and parameter count (18,184) as the graph models.
- **Physiologically informed graph** (47 edges). Edges represent established clinical relationships, grouped by function: circulation, respiration and acid–base, kidney, liver, blood and coagulation, electrolytes, and shock. These edges are fixed independently of the training data.
- **Statistical dependency graph** (91 edges). An edge connects two variables when the Pearson correlation of their jointly observed training values satisfies $|r| \geq 0.15$ and the pair was measured together at least 500 times. Imputed values are excluded, and the threshold was selected on dev utility.
- **Hybrid graph** and **fully connected graph** (561 edges). Not yet run; see *Remaining work* section.

Our primary model is a Graph Attention Network (GAT) [2], implemented using PyTorch Geometric [3]. GAT aggregates neighboring node representations through learned attention weights:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i) \cup i} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)}\right),$$

where $(\alpha_{ij}^{(l)})$ denotes the attention assigned to neighboring variable (j). This lets the model decide, for each patient-hour, which physiological relationships matter most. After two attention layers, we average the 34 node representations into a single vector, append the demographic and contextual features, and apply a linear layer to predict hourly sepsis risk. Because positive hours are rare, the loss up-weights them. We tuned the model's hyperparameters once, on the statistical graph, and kept them fixed for every graph, so that differences in results reflect graph structure alone.

As the non-graph baseline from our proposal, a window-feature XGBoost model [4] trained on the same split reaches 0.397 test utility. It uses richer temporal summary features than our node features, so we treat it as a reference point rather than a controlled comparison.

3.1 Remaining Work

- **Source-only transfer.** Refit normalization statistics, imputation medians, the class weight, and statistical edges on training patients from the source hospital alone, as proposed, and add the reverse direction (train on B, test on A).

- **Complete the graph comparison.** Train the hybrid graph (the union of clinical and statistical edges) and the fully connected graph under both the pooled and transfer regimes.
- **Multiple seeds.** Repeat each configuration over several seeds (likely 3-5) and report mean and standard deviation, so that we can tell whether small utility gaps are real.
- **Trend features.** Add the proposed 24-hour trend features to each node (e.g., slope, change since the previous measurement, and window minimum, maximum, and mean), since the direction of change in vital signs and labs is clinically informative.
- **Ablations and attention.** Remove physiological edge groups (e.g., shock and respiration) and examine learned attention weights, without assuming causal significance.
- **Architecture sensitivity.** Time permitting, compare GCN or GraphSAGE on the same graphs.

4 Partial Results

All code and data are available here.

Since the proposal, we have built a causal data pipeline and the GAT described above, constructed three of the four proposed graphs (no edges, statistical, and clinical), and compared them on the pooled split and in a first transfer experiment. Table 1 compares the graphs in two regimes: trained on both hospitals (pooled), and trained on Hospital A only, with epoch and threshold selected on dev patients from Hospital A, then scored on test patients from each hospital.

Table 1: GAT with fixed hyperparameters under different graph topologies (single seed).

		Pooled (train A+B)			Transfer (train A only)		
Graph	Edges	Dev U	Test U	Test AUPRC	Test A U	Test B U	Test B AUROC
Self-loops only	0	0.297	0.292	0.069	0.289	0.192	0.675
Clinical	47	0.308	**0.300**	**0.072**	0.283	0.194	**0.716**
Statistical	91	**0.318**	0.284	0.071	0.274	**0.198**	0.698
Hybrid	–	–	–	–	–	–	–
Fully connected	561	–	–	–	–	–	–

Graph structure. On the pooled split, the clinical graph has the best test utility (0.300) and AUPRC (0.072). The clinical and statistical graphs both reach 0.739 test AUROC versus 0.714 for self-loops, so message passing between variables mainly improves ranking. The statistical graph has the best dev utility (0.318) but the lowest test utility (0.284), and at its chosen threshold it raises more false alarms on test (8.70 per 100 non-septic hours versus 4.73 for self-loops).

Hospital transfer. Training on Hospital A alone costs every graph 0.07 to 0.10 utility on Hospital B relative to pooled training, while Hospital A changes little. On Hospital B, utility is nearly tied, but ranking quality orders as clinical (AUROC 0.716) $>$ statistical (0.698) $>$ self-loops (0.675). This is preliminary support for our hypothesis that clinical structure transfers better.

Limitations. All results are from a single seed, and differences of 0.01–0.02 utility may be within run-to-run variation. Hyperparameters were tuned on the statistical graph, which may favor it. The transfer experiment is not yet fully source-only: normalization statistics, imputation medians, and statistical edges were fit on training patients from both hospitals. The GAT also still trails the non-graph XGBoost reference (0.397).

References

[1] M. A. Reyna et al. *Early Prediction of Sepsis from Clinical Data: The PhysioNet/Computing in Cardiology Challenge 2019*. PhysioNet, 2019.

[2] P. Veličković et al. *Graph Attention Networks*. International Conference on Learning Representations, 2018.

[3] M. Fey & J. E. Lenssen. *Fast Graph Representation Learning with PyTorch Geometric*. ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.

[4] T. Chen & C. Guestrin. *XGBoost: A Scalable Tree Boosting System*. Proceedings of ACM SIGKDD, 2016.